

Advanced EDA Concepts: Combining Data, Multivariate Techniques, and Distance Metrics

CIS 635 — Knowledge Discovery & Data Mining

Part 2 of 3

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [12] this material when using or reproducing it.

Unit: From Data to Discovery — Part 2: Combining Data, Multivariate EDA, Similarity, and Applied Practice

This is Part 2 of a three-part reading. Part 1, **Basic EDA Concepts: Statistical Foundations, Data Preparation, and Exploratory Analysis Foundations**, covered the statistical vocabulary this reading assumes (populations and samples, levels of measurement, descriptive statistics, sampling bias, probability and Bayes' theorem), the main data-preparation techniques (missing-data mechanisms, encoding, scaling, integration, leakage), and univariate/bivariate EDA. Part 3, **Feature Selection and Dimensionality Reduction**, develops feature-selection and dimensionality-reduction techniques in full — this reading points there rather than covering them itself. This reading continues directly from Part 1.

Contents

1	Merging and Joining Tables	3
1.1	Why Combine Multiple Tables?	3
1.2	Keys: The Common Thread Between Tables	3
1.3	Merging Tables: Matching Rows by a Common Key	4
1.4	Merge vs. Join: What's the Difference?	5
1.5	A Second Example: Patients and Lab Results	5
2	Advanced Exploratory Data Analysis	5
2.1	Multivariate Exploration and Transformations	6
2.2	Group-wise, Conditional, and Interaction Analysis	6
2.3	High-Dimensional Data Exploration	6
2.4	Temporal and Spatial Patterns	7
2.5	Anomaly and Outlier Investigation	8
3	Similarity and Distance Measures	8
3.1	Distance Measures for Numerical Data	8
3.2	Cosine Similarity and Measures for Categorical Data	9
3.3	The Effect of Feature Scaling	9
3.4	High-Dimensional Data and the Curse of Dimensionality	10

3.5	Applications: Clustering, Nearest Neighbors, and Recommendation	10
4	Real-World Case Studies	11
4.1	Case Study: Early Warning for Sepsis from Structured Clinical Data	11
4.2	Case Study: Readmission Risk in a Large Diabetes Cohort	12
4.3	Case Study: Harmonizing In-Situ Soil Moisture Data for Precision Agriculture . . .	12
4.4	What the Three Case Studies Share	13
5	Interpretation and Communication of Findings	13
5.1	A Vocabulary of Increasing Claims	13
5.2	Communicating Uncertainty, Limitations, and Alternative Explanations	14
6	Common Pitfalls and Best Practices	14
6.1	Correlation, Causation, and Confounding	14
6.2	Sampling Bias, Selection Bias, and Missing-Data Problems	14
6.3	Misleading Visualizations and Overplotting	15
6.4	Statistical and Modeling Pitfalls	15
6.5	Ignoring Domain Knowledge and Cherry-Picking	16
6.6	Best Practices Summary	16
6.7	Recurring Misconceptions Worth Restating	16
7	Summary and Key Takeaways	16

Notation and Terminology Quick Reference

The table below collects the recurring terms this reading builds on, for quick lookup while reading or reviewing. Each term is developed fully in the section listed. Terms developed in Part 1 — population/sample, parameter/statistic, sampling bias, association/correlation, MCAR/MAR/MNAR, tidy data, and data leakage — and in Part 3 — filter/wrapper/embedded feature selection and dimensionality reduction — appear in those readings' own quick-reference tables.

Term	Short definition	§
Key (join key)	The shared column(s) used to match rows across two tables	1.2
Inner / left / right / outer join	The four rules for what happens to rows without a match	1.3
Curse of dimensionality	The breakdown of distance-based reasoning as dimensionality grows	3.4
Observation → causal claim	The ordered chain of increasingly strong analytical claims	5.1
Selection bias	A subset systematically different from the whole because of how it was included (compare sampling bias, Part 1, §1.5)	6.2

Learning Objectives

This reading assumes the statistical, preprocessing, and univariate/bivariate EDA foundations covered in Part 1. After completing this reading, you should be able to:

- Combine information stored across multiple related tables using common keys, and choose the appropriate type of merge or join for a given analytical question.
- Carry out multivariate, high-dimensional, temporal, and spatial exploratory data analysis in Python, building on the univariate and bivariate foundations from Part 1.
- Compute and interpret the major similarity and distance measures used in clustering, nearest-neighbor methods, and recommendation systems, and explain how dimensionality affects them.
- Trace a complete KDD pipeline — from a real-world problem, through data quality and preparation, EDA, and similarity reasoning, to a defensible finding — using real, documented case studies.
- Distinguish observation, pattern, association, correlation, hypothesis, evidence, and causal claim, and communicate uncertainty and limitations responsibly.
- Recognize the most common pitfalls in exploratory and preparatory data work and apply concrete practices that avoid them.

Part 3 of this reading, *Feature Selection and Dimensionality Reduction*, develops feature-selection strategies and dimensionality reduction in full.

1 Merging and Joining Tables

Real KDD projects rarely start with one tidy table. A hospital’s patient demographics live in one table, their lab results in another; a course’s student roster lives in one table, their grades in another. Before any of the multivariate or similarity-based techniques later in this reading can be applied across variables that live in different tables, those tables must first be combined into a single, analysis-ready dataset — exactly the kind of inter-related, joinable datasets this course’s project guidelines require (at least two datasets with a real relationship between them).

1.1 Why Combine Multiple Tables?

Splitting data across multiple tables is normal, and often deliberate: it avoids repeating a customer’s name on every single order they place, or a patient’s date of birth on every single lab result. Each table stores one *kind* of observational unit — exactly the tidy-data discipline introduced in Part 1, Section 2.6 — and the tables are linked through shared identifiers rather than by repeating information. Combining them only when needed, rather than storing everything in one giant table permanently, keeps the data smaller, more consistent, and easier to update.

1.2 Keys: The Common Thread Between Tables

A **key** is a column (or a small set of columns) whose values let you match a row in one table to the related row(s) in another. Consider two small tables:

Customers		Orders		
customer_id	name	order_id	customer_id	amount
1	Alice	101	1	\$50
2	Bob	102	1	\$20
3	Carol	103	2	\$75
		104	4	\$10

Here `customer_id` is the key: it uniquely identifies a row in **Customers** and appears again in **Orders** to say which customer placed each order. Matching only works when the key values line up exactly — "1" (text) will not match 1 (a number), and "Cust01" will not match "cust01".

Common Pitfall

The most common real-world reason a merge “silently” loses rows is a key-format mismatch: one table stores IDs as text with leading zeros ("0042"), the other as plain integers (42). Always check both key columns’ data types before merging.

1.3 Merging Tables: Matching Rows by a Common Key

Merging combines two tables by matching rows on a common key, and there are four basic ways to decide what happens to rows that *don't* find a match:

- **Inner:** keep only rows whose key appears in *both* tables.
- **Left:** keep every row from the first (left) table, filling in blanks (NaN) where no match exists in the second.
- **Right:** the mirror image of left — keep every row from the second table, filling in blanks where no match exists in the first.
- **Outer:** keep every row from *both* tables, filling in blanks wherever a match is missing on either side.

Merging **Customers** and **Orders** on `customer_id` with an **inner** join keeps only the rows where a customer actually placed an order:

customer_id	name	order_id	amount
1	Alice	101	\$50
1	Alice	102	\$20
2	Bob	103	\$75

Carol (customer 3, no orders) and order 104 (customer 4, not in the Customers table) both disappear — neither has a match on the other side. A **left** join would keep Carol, with NaN in her order columns; a **right** join would keep order 104, with NaN for the missing customer name; an **outer** join would keep both.

```
import pandas as pd

customers = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"]
})

orders = pd.DataFrame({
```

```

    "order_id": [101, 102, 103, 104],
    "customer_id": [1, 1, 2, 4],
    "amount": [50, 20, 75, 10]
})

# Inner join: only customers who actually placed an order
inner = pd.merge(customers, orders, on="customer_id", how="inner")

# Left join: every customer, even Carol who has no orders
left = pd.merge(customers, orders, on="customer_id", how="left")

print(inner)
print(left)

```

Key Idea

Think of an inner join as the *overlap* between two circles in a Venn diagram; a left or right join as one whole circle plus the overlap; and an outer join as both whole circles combined. Choosing the wrong one is a common, easy-to-miss source of silently dropped or unexpectedly duplicated data.

1.4 Merge vs. Join: What's the Difference?

In everyday use, “merging” and “joining” tables describe the *same underlying idea* — matching rows across tables on a key — and the two words are often used interchangeably. The small, practical difference in tools like pandas is *what gets matched by default*: `merge()` matches on one or more *columns* you specify (as in the example above), while `join()` is a convenience method that matches on the *row index* by default. Both support the same inner/left/right/outer choices described above; `join()` is simply the shorthand reached for when the key to match on is already the table's index rather than an ordinary column.

1.5 A Second Example: Patients and Lab Results

The same idea applies directly to the health-related datasets this course emphasizes. A **Patients** table (one row per patient, with demographics) and a **Lab Results** table (one row per test, linked by `patient_id`) combine exactly like Customers and Orders: an inner join keeps only patients who have at least one lab result; a left join keeps every patient, even those still awaiting their first test.

Tip

Before you merge, ask: What should happen to a patient with no lab results yet — should they disappear from the analysis (inner), or stay with blanks to be handled explicitly (left)? This is a data-analytic decision, not just a technical one, and it should be made deliberately rather than left to whatever the default happens to do.

2 Advanced Exploratory Data Analysis

Part 1, Section 3 established EDA as an analytical process built from univariate and bivariate building blocks, and Section 1 above showed how to combine multiple tables into one analysis-ready

dataset. This section develops the techniques needed once that combined, real-world dataset's genuine complexity — many variables, time structure, geography, and high dimensionality — is taken seriously.

2.1 Multivariate Exploration and Transformations

When more than two or three variables are of interest simultaneously, a full correlation matrix (often visualized as a heatmap) gives an overview of which pairs of numerical variables move together, while a pair plot arranges every pairwise scatter plot in a grid. Both techniques scale poorly past a few dozen variables, which is one motivation for the dimensionality-reduction techniques introduced in Part 3, Section 1.2. Data transformations — a log transform for a right-skewed variable, a Box-Cox or Yeo-Johnson transform more generally — are applied during EDA not only to prepare data for modeling but to make a relationship visible in the first place: a multiplicative relationship between two variables is often only apparent as a straight line once both axes are log-scaled.

2.2 Group-wise, Conditional, and Interaction Analysis

Group-wise analysis (Part 1, Section 3.4) generalizes to *conditional* analysis, where a relationship between two variables is examined separately within levels of a third: the correlation between medication count and readmission risk might differ substantially between patients under 65 and patients over 65, a phenomenon that a single, pooled correlation coefficient would average away entirely (this is the same underlying issue as Simpson's paradox, revisited in Section 6.1). An **interaction** exists when the effect of one variable genuinely depends on the level of another, and detecting interactions — typically through conditional plots or grouped summary statistics — is one of the main reasons multivariate EDA cannot be reduced to a sequence of independent univariate and bivariate checks.

```
# Interaction check: does the medication-count / readmission relationship
# differ between younger and older patients?
for age_group, sub in df.groupby("age_bucket"):
    corr = sub["num_medications"].corr(sub["readmitted_flag"])
    print(f"{age_group}: correlation = {corr:.3f}, n = {len(sub)}")
```

Key Idea

What question does this answer? Conditional analysis answers: “does the relationship between X and Y change depending on Z ?” *When is it appropriate?* Whenever a plausible confounding or effect-modifying variable is available. *How is it implemented?* By faceting a plot or grouping a summary statistic on Z before examining X and Y . *How should results be interpreted?* As evidence that a single pooled statistic may be misleading, not as proof that Z causes the difference. *What are its limitations?* Splitting on too many variables at once quickly reduces each subgroup to too few observations to draw any reliable conclusion from.

2.3 High-Dimensional Data Exploration

As the number of variables grows, pairwise techniques become impractical and geometric intuition starts to break down (a theme developed rigorously in Section 3.4). Practical high-dimensional EDA strategies include ranking variables by variance or by their correlation with an outcome of interest before visualizing only the most informative subset, and projecting the data onto a small

number of dimensions — most commonly with principal component analysis (PCA), developed intuitively in Part 3, Section 1.3 — so that the first two or three principal components can be plotted as a conventional scatter plot. A PCA plot of gene-expression samples, for instance, often reveals clustering by tissue type or disease subtype well before any formal classification model is built, making it a natural bridge to the similarity measures developed in Section 3.

2.4 Temporal and Spatial Patterns

Time-indexed data calls for its own EDA vocabulary: **trend** (long-run direction), **seasonality** (a pattern that repeats at a fixed period, such as daily or yearly), and **autocorrelation** (the degree to which a value is related to its own recent past — the same phenomenon Part 1, Section 1.7 illustrated with a soil-moisture sensor’s consecutive readings). Spatial data — readings tied to a geographic location — benefits from choropleth maps (shading regions by a summary statistic) and from explicitly checking whether nearby locations tend to have more similar values than distant ones (spatial autocorrelation), since ignoring geography can make a real regional pattern look like unstructured noise.

```
# Separating trend and seasonality from day-to-day noise
daily = df.set_index("date")["case_count"]

rolling_7day = daily.rolling(window=7, center=True).mean()
day_of_week_effect = daily.groupby(daily.index.dayofweek).mean()

print("7-day rolling trend (last 5 points):")
print(rolling_7day.tail())
print("\nAverage by day of week (reporting artifact check):")
print(day_of_week_effect)
```

Technique	Reveals	Caution
Correlation heatmap	Pairwise linear structure across many variables at once	Misses nonlinear relationships (Part 3, §1.4)
Pair plot / scatter-plot matrix	Every pairwise relationship simultaneously	Scales poorly beyond a few dozen variables
PCA projection	Dominant structure in high-dimensional data	Axes are combinations of features, not directly interpretable
Choropleth map	Regional patterns in a spatial variable	Raw counts must be rate-normalized for fair comparison
Small multiples (faceted plots)	How a relationship changes across a grouping variable	Too many facets make individual panels hard to read

Table 1: Advanced visualization techniques, what each is suited to reveal, and its main limitation.

Real-World Case Study

The Johns Hopkins CSSE COVID-19 dashboard [6] aggregated confirmed-case and mortality counts by geographic region and updated them continuously through the pandemic. Exploratory analysis of this kind of data had to handle several issues at once: strong day-of-week reporting artifacts (a temporal pattern that was a reporting artifact, not an epidemiological one), sudden jumps caused by retroactive batch corrections rather than genuine outbreaks (an anomaly that a naive outlier filter would have flagged incorrectly), and dramatic differences in testing capacity across regions that made raw case counts spatially incomparable without normalizing by a rate such as cases per 100,000 population. Every one of the temporal and spatial EDA cautions in this section applied directly to a dataset that millions of people, including public-health decision-makers, relied on in real time.

2.5 Anomaly and Outlier Investigation

Part 1, Section 2.4 distinguished invalid observations from legitimate outliers; EDA is where that distinction is actually investigated, using tools such as z-scores, the interquartile-range rule ($1.5 \times \text{IQR}$ beyond the first or third quartile), and, for multivariate data, distance-based methods that flag a point as anomalous only relative to its neighbors in feature space (foreshadowing Section 3). An outlier investigation is not complete once a point is flagged; it is complete once an analyst has determined *why* the point is unusual, since that determination decides whether it should be corrected, removed, or is in fact the most important observation in the dataset.

3 Similarity and Distance Measures

Many KDD tasks — clustering, nearest-neighbor classification, anomaly detection, recommendation — ultimately reduce to a single question: how similar are two records? This section develops the main ways of answering that question quantitatively. The Python examples below draw on scikit-learn [3], alongside NumPy [1] and seaborn [2] for the underlying array computation and visualization.

3.1 Distance Measures for Numerical Data

For two points $\mathbf{x} = (x_1, \dots, x_n)$ and $\mathbf{y} = (y_1, \dots, y_n)$ in n -dimensional feature space, the **Euclidean distance**

$$d_{\text{Euclidean}}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

is the straight-line distance and the most commonly used default. The **Manhattan distance** (or city-block distance)

$$d_{\text{Manhattan}}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

sums absolute differences along each dimension and is less sensitive to any single large difference than the squared term in Euclidean distance. Both are special cases of the **Minkowski distance**

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p},$$

which reduces to Manhattan distance at $p = 1$ and Euclidean distance at $p = 2$; larger values of p weight the single largest per-dimension difference more heavily.

Tip

Worked example. Consider two patients described by two standardized features (systolic blood pressure, heart rate): $\mathbf{x} = (1.2, 0.4)$ and $\mathbf{y} = (0.3, -0.6)$. The Euclidean distance is $\sqrt{(1.2 - 0.3)^2 + (0.4 - (-0.6))^2} = \sqrt{0.81 + 1.00} \approx 1.35$; the Manhattan distance is $|1.2 - 0.3| + |0.4 - (-0.6)| = 0.9 + 1.0 = 1.9$. Manhattan distance is larger here because it does not let a large per-dimension difference dominate the way squaring does in the Euclidean case — the two measures can, and often do, rank the same pair of points differently relative to a third point, which is precisely why the choice of measure in Section 3.1 is a modeling decision rather than an interchangeable default.

3.2 Cosine Similarity and Measures for Categorical Data

Cosine similarity measures the angle between two vectors rather than the distance between their endpoints,

$$\text{sim}_{\cos}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|},$$

and is preferred whenever the *direction* of a feature vector matters more than its magnitude — two documents with the same relative word-frequency profile but very different lengths should be judged similar, which cosine similarity captures and Euclidean distance does not. For purely categorical data, **Hamming distance** counts the number of positions at which two equal-length strings or category vectors differ, and is a natural fit for comparing, for example, two patients’ binary symptom-presence vectors.

Measure	Best suited to	Sensitive to
Euclidean	Continuous features on comparable scales	Feature scale; large single differences
Manhattan	Continuous features, especially high-dimensional or sparse	Feature scale, but less than Euclidean
Cosine similarity	Text/frequency vectors; magnitude irrelevant	Nothing about vector length; only direction
Hamming	Binary/categorical vectors of equal length	Exact mismatches only, no notion of “how different”

Table 2: Choosing a similarity or distance measure is a modeling decision, not an afterthought.

3.3 The Effect of Feature Scaling

Every distance measure in Section 3.1 treats each dimension as equally important by default. If one feature is measured in dollars (ranging into the hundreds of thousands) and another in years (ranging from 0 to 100), the dollar feature will dominate a Euclidean distance calculation regardless of its actual relevance, simply because of its larger numeric range. Part 1, Section 2.5 introduced the scaling and standardization techniques that address this; this is precisely why they are not optional preprocessing niceties but a prerequisite for any distance-based method to behave sensibly.

```

from sklearn.preprocessing import StandardScaler
from sklearn.metrics.pairwise import euclidean_distances

# Without scaling, "income" would dominate distance purely by magnitude
features = df[["age", "annual_income"]]
scaled = StandardScaler().fit_transform(features)

dist_matrix = euclidean_distances(scaled[:5])
print(dist_matrix.round(2))

```

3.4 High-Dimensional Data and the Curse of Dimensionality

As the number of dimensions grows, distance-based reasoning becomes progressively less useful: Beyer et al. [7] showed that, under broad conditions, the distance to the nearest neighbor and the distance to the farthest neighbor converge as dimensionality increases, so that the very notion of “nearest” loses discriminative power in sufficiently high dimensions. This is the formal version of the intuitive **curse of dimensionality**: as more features are added, data points become sparse relative to the volume of the space they occupy, and a fixed amount of data becomes an increasingly poor sample of that space. It is one of the strongest practical motivations for the feature selection and dimensionality reduction techniques developed in Part 3, and it is precisely the concern the vector-length arithmetic in the companion data-modalities reading was building toward.

Common Pitfall

Applying a standard Euclidean-distance clustering algorithm directly to a gene-expression matrix with thousands of genes as features, without any dimensionality reduction or feature selection, routinely produces clusters that reflect noise in the high-dimensional space rather than genuine biological structure. The curse of dimensionality is not a theoretical curiosity here; it is a direct, practical reason similarity-based methods and feature selection (Part 3) are usually applied together, not in isolation.

3.5 Applications: Clustering, Nearest Neighbors, and Recommendation

Clustering algorithms such as k -means and hierarchical clustering group records by minimizing within-cluster distance according to one of the measures above; the choice of measure directly shapes which clusters are found. Nearest-neighbor methods, including k -nearest-neighbor classification, label a new observation according to the majority label among its closest neighbors under a chosen distance measure. Recommendation systems frequently use cosine similarity between user or item feature vectors (a user’s rating profile, a product’s description) to identify the most similar items or users, precisely because relative preference pattern, not absolute rating magnitude, is what should drive a recommendation.

Real-World Case Study

A gene-expression dataset drawn from a public repository such as the NCBI Gene Expression Omnibus [10] typically records expression levels for thousands of genes across a few hundred tumor samples. Computing pairwise sample similarity directly on the raw, thousands-of-genes feature space runs directly into the curse of dimensionality described above; the standard

practice, consistent with Section 2.3, is to first reduce the feature space (via PCA, Part 3 Section 1.3, or by selecting the most variable genes, Part 3) before computing distances or applying clustering, so that the resulting sample groupings reflect genuine similarity in expression pattern rather than noise accumulated across thousands of largely uninformative dimensions.

4 Real-World Case Studies

The case studies embedded throughout Part 1 and in Sections 2–3 of this reading illustrated individual concepts in context. Public, well-documented resources of this kind are increasingly common in health-related KDD research; the MIMIC-III critical care database [4], for instance, has become a standard large-scale benchmark whose de-identified, integrated structure is typical of the tidy, multi-table clinical datasets Part 1, Section 2.6, and Section 1 of this reading, described. This section traces two complete pipelines end to end — problem, data, data quality, preparation, EDA, statistical reasoning, similarity/feature selection, findings, interpretation, and decision — to show how the pieces combine in practice.

4.1 Case Study: Early Warning for Sepsis from Structured Clinical Data

Problem. Sepsis is a life-threatening response to infection where early treatment substantially improves outcomes, but its early signs can be subtle and easy to miss amid a busy clinical workflow. **Data.** The Targeted Real-time Early Warning System (TREWS), evaluated across multiple hospitals, drew on structured electronic health record data — vital signs, laboratory results, and nursing documentation — rather than unstructured clinical notes, consistent with the observation in the companion data-modalities reading that structured fields are the modality most readily usable by a real-time monitoring system [11]. **Data quality and preparation.** Vital-sign streams from bedside monitors contain sensor artifacts and missing intervals when a monitor is disconnected or a patient is off the unit, and had to be cleaned and aligned to a common time base before any downstream reasoning could proceed reliably. **EDA and statistical reasoning.** Analysis of vital-sign and lab trajectories over time (Section 2.4) identified the trends and combinations of values associated with eventual sepsis onset, forming the statistical basis for a real-time risk score — and, as Part 1, Section 1.7 previewed, interpreting any such system’s alerts responsibly requires knowing the base rate of sepsis on the unit, not just the algorithm’s sensitivity. **Findings.** In a prospective, multi-site evaluation, patients whose sepsis was flagged by the system and confirmed by a clinician within three hours of the alert had significantly lower in-hospital mortality than those treated later, and earlier antibiotic administration was associated with the observed improvement [11]. **Interpretation and limitations.** The study design supports an association between earlier, system-prompted treatment and better outcomes; because clinicians retained full discretion over whether and how quickly to act on an alert, and because the evaluation was not a randomized trial, the finding is properly read as strong real-world evidence for the value of earlier detection and intervention, not as definitive proof that the algorithm’s alert itself, independent of the clinical response it prompted, caused the improvement — a distinction Section 5 returns to explicitly. **Decision-making.** The results supported continued clinical deployment of the early-warning system as a decision-support tool that prompts human review, rather than an autonomous decision-maker, illustrating a common and appropriate way KDD findings inform, rather than replace, professional judgment.

4.2 Case Study: Readmission Risk in a Large Diabetes Cohort

Problem. Hospitals face both clinical and financial incentives to reduce avoidable readmissions among patients with diabetes, but readmission risk depends on a mix of clinical, treatment, and process variables that are not obvious in advance. **Data.** The Diabetes 130-US Hospitals dataset [5] contains roughly 100,000 inpatient encounters across ten years and 130 hospitals, with fields covering demographics, diagnoses, medications, and whether HbA1c testing was performed, alongside the readmission outcome — exactly the kind of related encounter, diagnosis, and medication tables that Section 1’s merging techniques bring together into one analysis-ready table. **Data quality and preparation.** As discussed in Part 1, Section 2.7, several categorical fields have substantial missingness, encoded with a placeholder value rather than a proper missing-value marker (a common real-world data-quality issue introduced in Part 1, Section 3.2’s code example), diagnosis codes needed to be grouped into clinically interpretable categories, and encounter-level records for the same patient needed to be handled carefully to avoid the leakage risk of a naive record-level train/test split. **EDA.** Univariate and group-based analysis (Part 1, Section 3) showed substantial variation in readmission rates across admission types and primary diagnoses — the same kind of conditional-probability comparison worked out numerically in Part 1, Section 1.7 — while conditional analysis (Section 2.2) showed that the relationship between number of medications and readmission risk differed by age group, an interaction that a single pooled statistic would have missed. **Statistical reasoning and feature selection.** The original analysis found that patients who received HbA1c testing, particularly when the result led to a change in diabetes treatment, had a measurably different readmission pattern than patients who were not tested, motivating primary-care and quality-improvement recommendations around when this test should be ordered during a hospital stay [5]. Feature selection using both filter methods (association with the outcome) and embedded methods (feature importance from tree-based models) — developed fully in Part 3 — can further narrow the dozens of available fields to a manageable, clinically interpretable set for follow-on modeling. **Findings and limitations.** The association between HbA1c testing and readmission pattern is drawn from observational data, not a randomized trial, so hospitals with more thorough testing practices may also differ in other, unmeasured ways (staffing, overall care quality) that independently affect readmission; the finding is appropriately treated as motivation for a more targeted, prospective evaluation rather than as conclusive proof that ordering the test by itself reduces readmissions. **Decision-making.** The dataset has since become a standard benchmark for exactly this kind of careful, limitations-aware clinical predictive modeling, illustrating how a single well-documented real-world dataset can support both the original clinical question and years of subsequent methodological research.

4.3 Case Study: Harmonizing In-Situ Soil Moisture Data for Precision Agriculture

Problem. Satellite-based soil moisture products (such as NASA’s SMAP mission) provide broad spatial coverage but must be validated, and often calibrated, against ground-truth measurements before resource-constrained agricultural operations can rely on them for irrigation decisions. **Data.** The International Soil Moisture Network (ISMN) aggregates in-situ soil moisture measurements contributed by dozens of independent regional and national networks worldwide, each operating its own sensors, depths, and reporting conventions [9]. **Data quality and preparation.** This is a data-integration problem in the sense of Part 1, Section 2.6, and Section 1 of this reading, at a genuinely large scale: networks differ in sensor type and calibration, measurement depth, temporal resolution, and units, and ISMN applies standardized quality-control flagging to identify

implausible values (for instance, readings inconsistent with recent precipitation, or values outside a sensor’s physically plausible range) before harmonized data is released to users. **EDA.** Because soil moisture is inherently a temporal and spatial variable — exactly the kind of autocorrelated record Part 1, Section 1.7 used to illustrate non-independence — exploratory analysis of a given station’s record uses exactly the trend, seasonality, and spatial-comparability techniques developed in Section 2.4: a station’s readings must be interpreted relative to its local climate and soil type, not against a single global baseline. **Statistical reasoning and similarity.** Comparing a satellite product’s estimate against multiple nearby in-situ stations, and comparing stations to each other, relies on the same distance and correlation concepts from Part 1, Section 1.6, and Section 3 of this reading, since “how similar is this satellite estimate to ground truth” is itself a similarity question, and stations are often grouped by climate or soil-type similarity before a satellite product is evaluated separately within each group (an application of the conditional analysis introduced in Section 2.2). **Findings.** Harmonized, quality-controlled networks of this kind have been used extensively across the literature to validate and improve remote-sensing soil moisture products, precisely because a satellite estimate is only as trustworthy as the ground-truth data used to check it [9]. **Interpretation and limitations.** A validation result computed against stations concentrated in a particular region or climate does not automatically generalize to a different, poorly represented region — the same sampling-bias concern raised in Part 1, Section 1.5, here operating across geography rather than across a patient population. **Decision-making.** For a precision-agriculture application on a resource-constrained farm, this pipeline supports a concrete, practical decision: whether a satellite-derived soil moisture estimate is reliable enough, in a given region and season, to drive an irrigation recommendation directly, or whether it still requires local sensor calibration first.

4.4 What the Three Case Studies Share

Despite spanning critical care, hospital readmission, and precision agriculture, the three case studies above followed the same underlying shape: a real decision that needed support (whether to escalate care, whether to order a test, whether to trust a satellite estimate), data that arrived with domain-specific quality problems requiring preparation before it was usable (Part 1, Section 2), exploratory analysis that had to respect the data’s temporal, grouped, or spatial structure (Part 1, Section 3, and Section 2 of this reading) rather than treating every observation as independent, and a finding stated with an explicit account of what its study design could and could not establish (Section 5). None of the three relied on an exotic technique; all three relied on applying ordinary preprocessing, EDA, and interpretive discipline consistently, which is the central practical claim this reading is built around.

5 Interpretation and Communication of Findings

5.1 A Vocabulary of Increasing Claims

Responsible interpretation depends on keeping several related but distinct terms straight, ordered here from weakest to strongest claim: an **observation** is a single noted fact about the data (this patient’s length of stay was 9 days); a **pattern** is a regularity noticed across many observations (length of stay tends to be longer for a particular diagnosis); an **association** is a statistical relationship between two variables, established more formally than an informally noticed pattern; a **correlation** is the specific, quantified linear association introduced in Part 1, Section 1.6; a **hypothesis** is a specific, testable claim proposed to explain an observed pattern or association; **evidence** is data that bears on whether a hypothesis is more or less plausible; and a **causal**

claim asserts that changing one variable would, by itself, change another — the strongest and most demanding claim in this list, and the only one that generally requires either a randomized experiment or a carefully justified quasi-experimental design to support with confidence.

Appropriate	Inappropriate
“Patients who received HbA1c testing showed a different readmission pattern in this dataset.”	“HbA1c testing prevents readmission.”
“Earlier system-prompted treatment was associated with lower mortality in this multi-site evaluation.”	“The algorithm’s alert saved these patients’ lives.”
“Regions with lower testing capacity showed lower reported case counts.”	“Regions with lower testing capacity had fewer actual cases.”

Table 3: The same underlying finding, stated at the level of evidence it actually supports versus overstated as causal fact.

5.2 Communicating Uncertainty, Limitations, and Alternative Explanations

Responsible communication of a KDD finding routinely includes the sample size and source (so a reader can judge how representative it is), the specific population the finding applies to (Part 1, Section 1.1’s population/sample distinction, made explicit), any known data-quality caveats from the preparation stage, and at least one plausible alternative explanation for the observed pattern besides the one being proposed. This is not a matter of hedging every claim into meaninglessness; it is a matter of matching the confidence expressed in a conclusion to the strength of the study design and evidence that actually produced it, exactly as both case studies in Section 4 did when distinguishing a strong observational association from a causal claim their designs could not support.

6 Common Pitfalls and Best Practices

6.1 Correlation, Causation, and Confounding

The single most repeated caution in applied statistics is also the easiest to violate unintentionally: a correlation between two variables can arise because one causes the other, because both are caused by a third (confounding) variable, because of reverse causation, or simply by chance in a small or noisy sample. **Simpson’s paradox** — where a trend present in several groups of data disappears, or reverses, when the groups are combined — is a particularly stark illustration of why the conditional and group-based analysis introduced in Part 1, Section 3, and Section 2 of this reading, is not an optional refinement but often essential to avoid a directly misleading pooled conclusion.

6.2 Sampling Bias, Selection Bias, and Missing-Data Problems

Part 1, Section 1.5 introduced sampling bias; **selection bias** is the closely related problem of a subset of a dataset being systematically different from the whole because of how that subset came to be observed or included (survivorship bias — analyzing only patients who survived to discharge

Before finalizing a claim, state	Because
The sample size and source	Determines how much weight the finding can bear (Part 1, §1.1)
The population the sample is meant to represent	A finding about one hospital’s patients may not generalize to another’s (Part 1, §1.5)
Known data-quality caveats	A finding built on data with substantial MNAR missingness (Part 1, §2.2) needs a corresponding caveat
At least one alternative explanation	Guards against mistaking a confound for the proposed mechanism (§6.1)
The specific level of claim being made	Keeps the stated conclusion at the observation/pattern/association/causal level the evidence actually supports (§5.1)

Table 4: A brief checklist for stating a KDD finding responsibly, cross-referenced to where each concern is developed in full.

— is a common special case). Both interact with the missing-data mechanisms of Part 1, Section 2.2: MNAR missingness is, in effect, a form of selection bias operating within a single dataset.

6.3 Misleading Visualizations and Overplotting

A truncated y-axis can make a small difference look dramatic; a linear color scale on a highly skewed variable can hide most of the meaningful variation in a single color band; and **overplotting** — so many points on a scatter plot that they obscure each other — can hide the true density of a relationship, making a sparse region look as populated as a dense one. Each is avoidable with a deliberate choice (starting axes at zero when magnitude comparisons matter, using a log or perceptually uniform color scale, using transparency or 2D density plots for large point clouds).

6.4 Statistical and Modeling Pitfalls

Overinterpreting a p-value near the conventional 0.05 threshold as though it were a measure of effect size or practical importance, rather than a measure of how surprising a result would be under a specific null hypothesis, is a widely documented source of overstated findings; concerns of exactly this kind motivated broad calls for more cautious interpretation of statistical significance across the empirical sciences [8]. The **multiple-comparisons problem** compounds this: testing dozens of variables for a significant association with an outcome all but guarantees that some will appear significant by chance alone, and reporting only the ones that did (without correcting for the number of tests performed) is a well-documented path to a finding that will not replicate. **Data leakage** and **feature-selection bias** (Part 1, Section 2.7, and Part 3, Section 1.5) belong in the same family: each makes a model or finding look better in evaluation than it will perform in practice. Applying an **inappropriate distance measure** — Euclidean distance on unscaled features, or on data whose curse of dimensionality (Section 3.4) has not been addressed — silently produces clusters or neighbor relationships that do not reflect the intended notion of similarity at all.

6.5 Ignoring Domain Knowledge and Cherry-Picking

A purely data-driven analysis that ignores available domain knowledge risks treating a known artifact of measurement (a hospital’s coding practice, a sensor’s calibration schedule) as a genuine finding. **Cherry-picking** — reporting only the subset of analyses, time windows, or variable definitions that produced the most compelling result, without disclosing the others that were tried — is one of the more subtle threats to reproducibility, precisely because the final report can look entirely rigorous in isolation while omitting the exploratory process that selected it.

Common Pitfall

The frequently repeated “beer and diapers” market-basket anecdote — that a supermarket chain discovered, through data mining, that customers who bought diapers also tended to buy beer, and increased sales by placing them near each other — is widely cited in data mining courses and textbooks but is poorly sourced, with no verifiable, publicly documented original study behind the specific claim as usually retold. It is included here deliberately, as a caution rather than an example to emulate: before repeating a memorable data-mining anecdote, or citing your own analysis as a definitive finding, ask what you would need to know about the original study design, sample, and evaluation before treating it as established fact.

6.6 Best Practices Summary

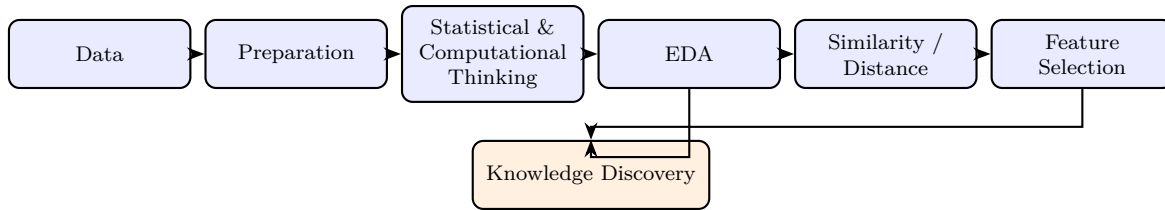
The practices that address the pitfalls above are cumulative, not a separate checklist: document every preprocessing decision and its rationale (Part 1, Section 2.7); check that a merge or join is not silently dropping or duplicating rows before trusting anything downstream of it (Section 1); visualize before and alongside computing summary statistics (Part 1, Section 3.5); check for interactions and confounding with conditional analysis before pooling a result (Section 2.2); perform feature selection inside, never outside, a cross-validation fold (Part 3, Section 1.5); state findings at the level of evidence the study design actually supports (Section 5); and treat a striking or convenient result with the same scrutiny applied to an inconvenient one.

6.7 Recurring Misconceptions Worth Restating

A handful of misconceptions recur often enough, across otherwise careful analyses, to be worth restating plainly. *A large sample does not fix a biased sample*: doubling the size of a sample drawn through a biased mechanism produces a more precisely estimated wrong answer, not a correct one (Part 1, Section 1.5). *Statistical significance is not the same as practical importance*: a very large sample can make a trivially small, practically meaningless difference statistically significant (Section 6.4). *An outlier is not automatically an error*, and an error is not automatically an outlier — the two require the domain-informed investigation described in Section 2.5, not a single automated rule applied uniformly. *A correlation coefficient of zero does not mean two variables are unrelated*; it means they are not *linearly* related, and a clear nonlinear pattern can coexist with $r \approx 0$ (Part 1, Section 1.6). *Removing a sensitive feature from a dataset does not automatically remove the information it carried*, if a closely correlated proxy feature remains available to a model (Part 3, Section 1.5).

7 Summary and Key Takeaways

All three readings in this unit converge on the following takeaways.



1. **Every dataset is a sample standing in for an unknown population, and every summary statistic is an estimate, not a fact.** Sampling bias and missing-data mechanisms shape what a dataset can and cannot support before any analysis technique is even chosen.
2. **Preprocessing decisions are analytical decisions.** How missing values, outliers, categorical encodings, and even how multiple tables are merged or joined are handled changes which patterns EDA finds and which features a model ultimately relies on.
3. **EDA is a chain of analytical questions, not a sequence of charts.** Univariate, bivariate, and multivariate techniques build on each other, and conditional analysis is often what reveals or corrects a misleading pooled result.
4. **Similarity is not a single, universal notion.** The right distance or similarity measure depends on data type, scaling, and dimensionality, and high-dimensional data requires feature selection or dimensionality reduction before distance-based reasoning is meaningful.
5. **Feature selection and EDA inform each other.** Filter, wrapper, and embedded methods each make different trade-offs between speed, model-awareness, and leakage risk — developed fully in Part 3.
6. **Real KDD findings live on a spectrum from observation to causal claim,** and responsible interpretation states a finding at the level of evidence its data and design actually support — exactly the discipline both worked case studies in this reading demonstrated.
7. **Nearly every common pitfall in applied data analysis — confounding, selection bias, misleading visualization, leakage, cherry-picking — has a concrete, learnable countermeasure,** and applying them consistently, not merely knowing about them, is what separates a defensible KDD finding from an anecdote.

Discussion Questions

1. You are given a Students table and an Enrollments table linked by `student_id`. A student who has not yet enrolled in any course appears in Students but not in Enrollments. Which type of join would you use to list every student along with their courses (if any), and which type would you use to list only students who are currently enrolled in something? Explain your reasoning.
2. Choose one case study from Section 4 and identify one point in its pipeline (data quality, preparation, table-combination, or interpretation) where a different, still-reasonable decision could plausibly have changed the reported finding. What does this suggest about how much a single KDD result should be trusted on its own?
3. Section 6 argues that data leakage and feature-selection bias are dangerous specifically because they make results look *better*, not worse. Why might this make them harder to catch in practice than an error that simply produces an obviously wrong answer?

4. This reading distinguishes sampling bias (Part 1, Section 1.5) from selection bias (Section 6.2) as closely related but not identical. Construct one example of each, and explain what would need to be true about how the data was collected to tell them apart.
5. Revisit the “beer and diapers” warning box in Section 6.5. What specific pieces of information would you need before you could responsibly cite a similar, widely repeated data-mining anecdote in your own work?

References

- [1] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [2] Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>
- [3] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [4] Johnson, A. E. W., Pollard, T. J., Shen, L., et al. (2016). MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3, 160035. <https://doi.org/10.1038/sdata.2016.35>
- [5] Strack, B., DeShazo, J. P., Gennings, C., et al. (2014). Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records. *BioMed Research International*, 2014, Article ID 781670. <https://doi.org/10.1155/2014/781670>
- [6] Dong, E., Du, H., & Gardner, L. (2020). An interactive web-based dashboard to track COVID-19 in real time. *The Lancet Infectious Diseases*, 20(5), 533–534. [https://doi.org/10.1016/S1473-3099\(20\)30120-1](https://doi.org/10.1016/S1473-3099(20)30120-1)
- [7] Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When is “nearest neighbor” meaningful? In *Proceedings of the 7th International Conference on Database Theory (ICDT 1999)*, Lecture Notes in Computer Science, vol. 1540, 217–235. Springer. https://doi.org/10.1007/3-540-49257-7_15
- [8] Ioannidis, J. P. A. (2005). Why most published research findings are false. *PLoS Medicine*, 2(8), e124. <https://doi.org/10.1371/journal.pmed.0020124>
- [9] Dorigo, W., Himmelbauer, I., Aberer, D., et al. (2021). The International Soil Moisture Network: Serving Earth system science for over a decade. *Hydrology and Earth System Sciences*, 25(11), 5749–5804. <https://doi.org/10.5194/hess-25-5749-2021>
- [10] Barrett, T., Wilhite, S. E., Ledoux, P., et al. (2013). NCBI GEO: Archive for functional genomics data sets — update. *Nucleic Acids Research*, 41(D1), D991–D995. <https://doi.org/10.1093/nar/gks1193>
- [11] Adams, R., Henry, K. E., Sridharan, A., et al. (2022). Prospective, multi-site study of patient outcomes after implementation of the TREWS machine learning-based early warning system for sepsis. *Nature Medicine*, 28, 1455–1460. <https://doi.org/10.1038/s41591-022-01894-0>
- [12] Hasan, K. (2026). *Knowledge Discovery and Data Mining*. Grand Valley State University. Available at: https://github.com/mdkamrulhasan/data_mining_kdd

Pitfall	Where discussed	Primary countermeasure
Correlation mistaken for causation	§6.1	Identify plausible confounders; prefer experimental or quasi-experimental designs for causal claims
Simpson’s paradox / hidden confounding	§6.1, §2.2	Condition on relevant subgroups before pooling a result
Sampling / selection bias	Part 1 §1.5, §6.2	Document how the sample was obtained; check representativeness against the target population
MNAR missing data	Part 1 §2.2	Investigate <i>why</i> data is missing before choosing an imputation strategy
Silently dropped or duplicated rows	§1.3	Check row counts before and after every merge or join
Misleading visualization, overplotting	§6.3	Zero-based axes for magnitude claims; transparency or density plots for large point clouds
Overinterpreted significance, multiple comparisons	§6.4	Report effect sizes alongside p-values; correct for the number of tests performed
Data leakage	Part 1 §2.7, §6.4	Fit every preprocessing and selection step only within the training fold
Feature-selection bias	Part 3 §1.5, §6.4	Select features inside cross-validation, never on the full dataset first
Inappropriate distance measure	§3.4, §6.4	Match the measure to data type and scale; reduce dimensionality first when needed
Cherry-picking, unreported analyses	§6.5	Pre-register or disclose all analyses attempted, not only the reported one

Table 5: A quick-reference map from pitfall to countermeasure, cross-referenced to where each is discussed in full.